

Getting Started with Python

1. Output `Hello World` message
2. Data Types
 - integer
 - float
 - Boolean
 - string
3. Variables
4. Lists
5. Dictionaries
6. If Statement
7. For-Loop
8. Functions
9. Modules

1. Output `Hello World` message

```
In [1]: # printing my first message  
print('Hello World')
```

Hello World

2. Data Types

```
In [2]: print(type(1))
```

<class 'int'>

```
In [3]: print(type(2.4))
```

<class 'float'>

```
In [4]: print(1 + 2)
```

3

```
In [5]: print("4-1")
```

4-1

```
In [6]: print(type(True))
```

<class 'bool'>

```
In [7]: print(type(False))
```

<class 'bool'>

```
In [8]: print(type('Hello World'))
```

```
<class 'str'>
```

3. Variables

```
In [9]: message_1 = "I'm learning Python"
print("message_1=", message_1)
```

```
message_1= I'm learning Python
```

```
In [10]: message_2 = "and it's fun"
print(message_2)
```

```
and it's fun
```

```
In [11]: message = message_1 + " " + message_2
print(message)
```

```
I'm learning Python and it's fun
```

```
In [12]: # Another way to formate the output
print(f'message_1: {message_1}\nmessage_2: {message_2}')
```

```
message_1: I'm learning Python
```

```
message_2: and it's fun
```

4. Lists

- Adding elements to a list
- Removing elements from a list
- Accessing elements in a list
- Slicing a list
- Insert an element into a list
- Concatenating lists
- Nested lists
- Sorting a list
- Copying a list

```
In [13]: countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries)
```

```
# add an element to the list
countries.append('Japan')
print(countries)
```

```
# remove an element from the list
countries.remove('Brazil')
print(countries)
```

```
['United States', 'India', 'Taiwan', 'Brazil']
```

```
['United States', 'India', 'Taiwan', 'Brazil', 'Japan']
```

```
['United States', 'India', 'Taiwan', 'Japan']
```

```
In [14]: # More ways to remove items from a list

countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries)

# Remove the last item
removed_country = countries.pop()
print(removed_country)

countries.pop(-1)
print(countries)

del countries[0]
print(countries)
```

```
['United States', 'India', 'Taiwan', 'Brazil']
Brazil
['United States', 'India']
['India']
```

```
In [15]: # access an element in a list (zero indexed)
countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[0])
```

```
United States
```

```
In [16]: # reverse indexed
countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[-1])
```

```
Brazil
```

```
In [17]: countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[-4])
```

```
United States
```

```
In [18]: # slicing a list[start:stop]
countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[0:2])
```

```
['United States', 'India']
```

```
In [19]: countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[1:])
```

```
['India', 'Taiwan', 'Brazil']
```

```
In [20]: countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[:2])
```

```
['United States', 'India']
```

```
In [21]: # insert an element in the list
countries = ['United States', 'India', 'Taiwan', 'Brazil']
countries.insert(0, 'Spain')
print(countries)
```

```
['Spain', 'United States', 'India', 'Taiwan', 'Brazil']
```

```
In [22]: # Concatenation of lists
countries = ['United States', 'India', 'Taiwan', 'Brazil']
countries_2 = ["Finland", "Sweden", "Denmark", "Norway", "Iceland"]
print(countries + countries_2)
```

```
['United States', 'India', 'Taiwan', 'Brazil', 'Finland', 'Sweden',
'Denmark', 'Norway', 'Iceland']
```

```
In [23]: # nested list of lists
countries = ['United States', 'India', 'Taiwan', 'Brazil']
countries_2 = ['China', 'Japan', 'South Korea', 'Vietnam']
nested_list = [countries, countries_2]
print(nested_list)
```

```
[['United States', 'India', 'Taiwan', 'Brazil'], ['China', 'Japan',
'South Korea', 'Vietnam']]
```

```
In [24]: # sorting a list
numbers = [4, 3, 10, 9, 8, 2]
print(numbers)
numbers.sort()
print(numbers)
```

```
[4, 3, 10, 9, 8, 2]
[2, 3, 4, 8, 9, 10]
```

```
In [25]: # sorting in reverse order
numbers = [4, 3, 10, 9, 8, 2]
numbers.sort(reverse=True)
print(numbers)
```

```
[10, 9, 8, 4, 3, 2]
```

```
In [26]: # update an element in the list
numbers = [4, 3, 10, 9, 8, 2]
print(numbers)

numbers[0] = 1000
print(numbers)
```

```
[4, 3, 10, 9, 8, 2]
[1000, 3, 10, 9, 8, 2]
```

```
In [27]: # copy a list
countries = ['United States', 'India', 'Taiwan', 'Brazil']
new_list = countries[:]
print(new_list)
```

```
['United States', 'India', 'Taiwan', 'Brazil']
```

```
In [28]: # another way to copy a list
new_list2 = countries.copy()
print(new_list2)
```

```
['United States', 'India', 'Taiwan', 'Brazil']
```

5 Dictionaries

1. Copy a dictionary
2. Remove an element from dictionary

```
In [29]: my_dict = {'key1': 'value1', 'key2': 'value2'}
print(my_dict)
```

```
{'key1': 'value1', 'key2': 'value2'}
```

```
In [30]: my_data = {'name': 'Michael', 'age': 63}
print(my_data)
```

```
{'name': 'Michael', 'age': 63}
```

```
In [31]: print(my_data.keys())
```

```
dict_keys(['name', 'age'])
```

```
In [32]: print(my_data.values())
```

```
dict_values(['Michael', 63])
```

```
In [33]: my_data.items()
```

```
Out[33]: dict_items([('name', 'Michael'), ('age', 63)])
```

```
In [34]: my_data['height'] = 165
print(my_data)
```

```
{'name': 'Michael', 'age': 63, 'height': 165}
```

```
In [35]: my_data.update({'height': 180})
print(my_data)
```

```
{'name': 'Michael', 'age': 63, 'height': 180}
```

5.1 Copy a dictionary

```
In [36]: my_data = {'name': 'Michael', 'age': 63, 'height': 165}
print(my_data)
```

```
new_dict = my_data.copy() # copy by data
print(new_dict)
```

```
new_dict2 = my_data # copy by reference
new_dict2
```

```
{'name': 'Michael', 'age': 63, 'height': 165}
```

```
{'name': 'Michael', 'age': 63, 'height': 165}
```

```
Out[36]: {'name': 'Michael', 'age': 63, 'height': 165}
```

```
In [37]: my_data.update({'height': 180})
print(my_data)
print(new_dict) # does not affect the other copy of data
new_dict2 # reference to the same copy of data
```

```
{'name': 'Michael', 'age': 63, 'height': 180}  
{'name': 'Michael', 'age': 63, 'height': 165}
```

```
Out[37]: {'name': 'Michael', 'age': 63, 'height': 180}
```

5.2 Remove an element from dictionary

```
In [38]: my_data = {'name': 'Michael', 'age': 63, 'height': 180}  
         print(my_data)  
         my_data.pop('name') # pop by key  
         my_data
```

```
{'name': 'Michael', 'age': 63, 'height': 180}
```

```
Out[38]: {'age': 63, 'height': 180}
```

```
In [39]: del my_data['age'] # delete by key  
         my_data
```

```
Out[39]: {'height': 180}
```

```
In [40]: my_data.clear() # remove all elements  
         my_data
```

```
Out[40]: {}
```

6. If Statement

Syntax:

```
if (condition):  
    code block  
else:  
    code block
```

```
In [41]: age = 10  
  
         if age >= 18:  
             print("You're an adult!")  
         elif age >= 13:  
             print("You're a teenager.")  
         else:  
             print("You're a kid.")
```

You're a kid.

7 For-loop

Syntax:

```
for i in range(10):  
    print(i)
```

```
In [42]: countries = ['United States', 'India', 'Taiwan', 'Brazil']  
  
countries
```

```
Out[42]: ['United States', 'India', 'Taiwan', 'Brazil']
```

```
In [43]: for country in countries:  
         print(country)
```

```
United States  
India  
Taiwan  
Brazil
```

```
In [44]: for i, country in enumerate(countries):  
         print(i)  
         print(country)
```

```
0  
United States  
1  
India  
2  
Taiwan  
3  
Brazil
```

```
In [45]: my_data = {'name': 'Michael', 'age': 63, 'height': 180}  
  
my_data
```

```
Out[45]: {'name': 'Michael', 'age': 63, 'height': 180}
```

```
In [46]: for key, value in my_data.items():  
         print(key)  
         print(value)
```

```
name  
Michael  
age  
63  
height  
180
```

8. Functions

1. Built-in functions
2. User-defined Functions

Syntax:

```
def function():  
    <code>  
    return
```

```
In [47]: def add(a, b):  
         return a+b
```

```
In [48]: add(1,3)
```

```
Out[48]: 4
```

8.1 Built-in Functions

```
In [49]: countries = ['United States', 'India', 'Taiwan', 'Brazil']  
  
countries
```

```
Out[49]: ['United States', 'India', 'Taiwan', 'Brazil']
```

```
In [50]: len(countries)
```

```
Out[50]: 4
```

```
In [51]: max([10,3, 6, 33, 100])
```

```
Out[51]: 100
```

```
In [52]: min([10,3, 6, 33, 100])
```

```
Out[52]: 3
```

```
In [53]: type(countries)
```

```
Out[53]: list
```

```
In [54]: print('Hello World'.upper())  
         print('Hello World'.lower())  
         print('Hello World'.title())  
         print('Hello World'.count('l'))  
         print('Hello World'.replace('o', '0'))
```

```
HELLO WORLD  
hello world  
Hello World  
3  
Hell0 W0rld
```

8.2 User-defined Functions

```
In [55]: import psutil  
  
memory = psutil.virtual_memory()  
  
def convert_bytes(size):  
    # Convert bytes to GB  
    gb = size / (1024 ** 3)
```



```

    return gb

total_gb = convert_bytes (memory.total)
available_gb = convert_bytes(memory.available)
used_gb = convert_bytes (memory.used)

print(f"Total RAM: {total_gb:.3f} GB ")
print(f"Available RAM: {available_gb:.3f} GB ")
print(f"Used RAM: {used_gb:.3f} GB ")
print(f"RAM Usage: {memory.percent}%")

```

Total RAM: 16.000 GB
 Available RAM: 5.561 GB
 Used RAM: 7.390 GB
 RAM Usage: 65.2%

9 Modules

1. OS Module
2. Psutil Module (See User-defined Functions above)a

Syntax:

```
import <module_name>
```

9.1 OS Module

```
import os
```

```
In [56]: import os
```

```
In [57]: os.getcwd()
```

```
Out[57]: '/Users/michaelkao/My_Projects/Python'
```

```
In [58]: os.listdir()
```

```
Out[58]: ['graphUSA50.py',
          'my_module.py',
          'hello.py',
          '2. Python Tips and Tricks.pdf',
          'sortDict.py',
          'genExp.py',
          'biTree.py',
          'multiprocessing1.py',
          'process_images_1P.py',
          'csvEx2.py',
          'xmlEx.py',
          'arithmetic.py',
          'tkinterEx2.py',
          '1. Python Tutorial TW.ipynb',
          'randomWalk.py',
          'New Folder',
```

```
'video',
'gameSquareDays.py',
'unit_test',
'biTree_v1.py',
'2. Python Tips and Tricks.png',
'tupleEx1.py',
'2. Python Tips and Tricks TW.ipynb',
'process_images_CT.py',
'.DS_Store',
'primeLib.py',
'osModule.py',
'smsViaHTTP.py',
'bstEx2.py',
'setEx.py',
'csvEx.py',
'dictEx.py',
'logger',
'gameGuess4.py',
'guiEx.py',
'bsTreeEx.py',
'requirements.txt',
'heapBinary.py',
'uv.lock',
'perfComprehensions.py',
'generatorEx.py',
'employeeEx.py',
'perfHeaps.py',
'loopEx.py',
'process_images_CP.py',
'2. Python Tips and Tricks.html',
'smsViaSMTP.py',
'dunderEx.py',
'test_copy.txt',
'fibo.py',
'bsTree.py',
'loggingEx.py',
'eafp.py',
'jsonEx2.py',
'modEx3.py',
'iterEx2.py',
'xmlEx4.py',
'funcEx.py',
'sysPython.py',
'gameRollDice.py',
'bst.py',
'randomEx.py',
'filesEx6.py',
'graphEx5.py',
'piEstimate.py',
'gcd.py',
'slicingEx.py',
'gRatio.py',
'stairs.py',
'pyproject.toml',
'filesEx2.py',
'quake.py',
```

```
'perf_LvsT.py',
'gameSnake3.py',
'factorialEx.py',
'emailSendHTML.py',
'output',
'sortDict2.py',
'biTreeEx_v2.py',
'emailOnQuake.py',
'multiprocess10.py',
'2. Python Tips and Tricks.ipynb',
'filesEx3.py',
'gameSnake2.py',
'pingSvr.py',
'perfDB.py',
'modEx2.py',
'iterEx3.py',
'__pycache__',
'sortObj.py',
'xmlEx5.py',
'piLib.py',
'greetings2.py',
'graphEx4.py',
'iterEx.py',
'bstEx.py',
'emailSendAttach.py',
'perfListvsGenerator.py',
'iterEx4.py',
'graphEx.py',
'listEx.py',
'README.md',
'biTreeEx_v1.py',
'xmlEx2.py',
'graphEx3.py',
'gameSRP.py',
'iterEx4_noflag.py',
'greetings.py',
'gameGuessNbr.py',
'SQLiteEx.py',
'jsonEx.py',
'scheduleEx.py',
'4. Python Advanced Topics2.ipynb',
'cacheEx2.py',
'people.py',
'brAsyncEx.py',
'1. Python Tutorial.pdf',
'3. Python Advanced Topics1.ipynb',
'.gitignore',
'heapBinomial.py',
'gameSnake5.py',
'condFlowEx.py',
'qrCodeGen.py',
'jsonAPI.py',
'.env',
'magicSquare.py',
'my_moduleEx.py',
'cacheEx.py',
```

```
'filesEx5.py',
'perfListvsTuple.py',
'graphEx6.py',
'.venv',
'gameSnake4.py',
'iterEx5.py',
'emailPublisher.py',
'modEx4.py',
'sample',
'.python-version',
'qt5sql.py',
'xmlEx3.py',
'brSyncEx.py',
'Python.code-workspace',
'graphEx2.py',
'xmlInspect.py',
'filesEx1.py',
'ProductionPythonCode',
'dates.py',
'go2Mars.py',
'img_processed',
'meminfo.py',
'emailSender.py',
'sortTuple.py',
'qt5Ex1.py',
'httpPing.py',
'gamePoker.py',
'environment.yaml',
'legbEx.py',
'tupleEx3.py',
'comprehensionEx.py',
'fastapiEx.py',
'.git',
'main.py',
'legbEx3.py',
'flagsEx.py',
'numberConversion.py',
'reEx.py',
'glogTime.py',
'biTreeEx.py',
'smsOnQuake.py',
'video-orig',
'comprehensionEx2.py',
'reEmails.py',
'data',
'advFormatting.py',
'csv_processor.log',
'template.env.txt',
'multiprocessing2.py',
'legbEx2.py',
'lambdaEx.py',
'multiprocessingEx.py',
'sieveEra.py',
'photo',
'1. Python Tutorial.html',
'tkinterEx1.py',
```

```
'sortList.py',  
'renameEx.py',  
'git',  
'gameSnake.py',  
'1. Python Tutorial.ipynb',  
'mcp_server.py',  
'mem_profile.py',  
'heapFibo.py',  
'daxia_rice_noodle_sign.py',  
'biTree_v2.py',  
'cacheLib.py',  
'tupleEx2.py',  
'exceptionEx.py']
```

```
In [59]: # make a directory with (name, mode, exist_ok)  
os.makedirs("New Folder", exist_ok=True)  
os.listdir("./New Folder")
```

```
Out[59]: []
```